

IMPORTING LIBRARIES AND MANAGING DATASET

```
import re
```

```
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
```

```
!pip install tensorflow-addons
import numpy as np
from numpy import save
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import os
import zipfile
import csv
import cv2
import shutil
import random
import seaborn as sns
from PIL import Image, ImageEnhance
import imgaug.augmenters as iaa
from tqdm import tqdm
import tensorflow as tf
import tensorflow_addons as tfa
from tensorflow import keras
from keras.callbacks import EarlyStopping
from tensorflow.keras import Model
from tensorflow.keras import layers
from tensorflow.keras.models import Model, Sequential
from keras.utils.vis_utils import plot_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout, Dense, Flatten, BatchNormalization
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers import RMSprop

from tensorflow.keras.applications.inception_v3 import InceptionV3
from tensorflow.keras.applications.resnet import ResNet50
from tensorflow.keras.applications import vgg16
```

Looking in indexes: <https://pypi.org/simple>, <https://us-python.pkg.dev/colab-wheels/public/simple/>

Collecting tensorflow-addons

Downloading tensorflow-addons-0.20.0-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (591 kB)
 591.0/591.0 kB 14.2 MB/s eta 0:00:00

Collecting typeguard<3.0.0,>=2.7

Downloading typeguard-2.13.3-py3-none-any.whl (17 kB)

Requirement already satisfied: packaging in /usr/local/lib/python3.9/dist-packages (from tensorflow-addons) (23.0)

Installing collected packages: typeguard, tensorflow-addons

Successfully installed tensorflow-addons-0.20.0 typeguard-2.13.3

/usr/local/lib/python3.9/dist-packages/tensorflow_addons/utils/tfa_eol_msg.py:23: UserWarning:

TensorFlow Addons (TFA) has ended development and introduction of new features.

TFA has entered a minimal maintenance and release mode until a planned end of life in May 2024.

Please modify downstream libraries to take dependencies from other repositories in our TensorFlow community (e.g. Ker

For more information see: <https://github.com/tensorflow/addons/issues/2807>

```
warnings.warn(
```

```
from keras import regularizers
from tensorflow.keras.optimizers import Adam
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
device_name = tf.test.gpu_device_name()
if "GPU" not in device_name:
    print("GPU device not found")
print('Found GPU at: {}'.format(device_name))
```

```
Found GPU at: /device:GPU:0
```

```
import warnings
warnings.simplefilter(action="ignore", category=FutureWarning)
```

```
os.getcwd()
```

```
'/content'
```

```
os.chdir('/content/drive/MyDrive/ODR')
os.getcwd()
```

```
'/content/drive/MyDrive/ODR'
```

```
skip_folders = ["hypertension", "diabetes", "others"]

image_list = []
disease_name = []
for root, dirs, files in os.walk("./datasets"):
    if os.path.basename(root) in skip_folders:
        continue
    for file in files:
        if file.endswith(".png"): # or whatever file format you have
            image_path = os.path.join(root, file)
            class_name = os.path.basename(root)
            if class_name not in disease_name:
                disease_name.append(class_name)
            image_list.append((image_path, class_name))

df = pd.DataFrame(image_list, columns=["image_path", "class_name"])
```

```
df
```

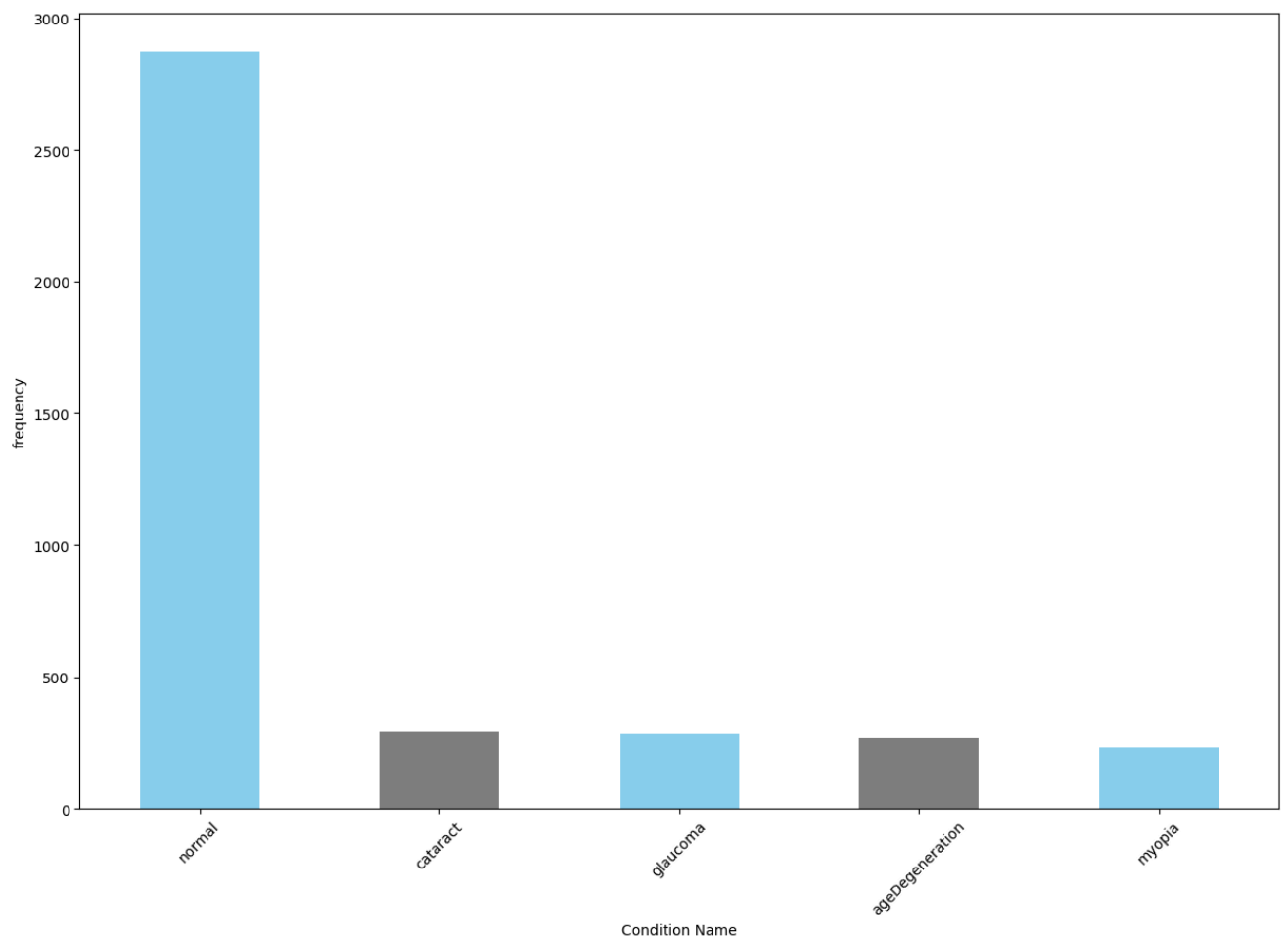
	image_path	class_name
0	./datasets/glaucoma/image0.png	glaucoma
1	./datasets/glaucoma/image1.png	glaucoma
2	./datasets/glaucoma/image104.png	glaucoma
3	./datasets/glaucoma/image100.png	glaucoma
4	./datasets/glaucoma/image101.png	glaucoma
...	...	...
3943	./datasets/myopia/image96.png	myopia
3944	./datasets/myopia/image91.png	myopia
3945	./datasets/myopia/image98.png	myopia
3946	./datasets/myopia/image97.png	myopia
3947	./datasets/myopia/image99.png	myopia

3948 rows x 2 columns

```
disease_name
```

```
['glaucoma', 'normal', 'cataract', 'ageDegeneration', 'myopia']
```

```
fig = plt.figure(figsize=(15, 10))
df['class_name'].value_counts().plot(kind='bar', xlabel='Condition Name', ylabel='frequency', color=['skyblue', 'gray'],
plt.xticks(rotation=45)
plt.show()
plt.savefig('BeforeSolvingClassImbalance.png')
```



<Figure size 640x480 with 0 Axes>

```
df['class_name'].value_counts()
```

```
normal      2873
cataract    293
glaucoma    284
ageDegeneration 266
myopia      232
Name: class_name, dtype: int64
```

```
df['class_name'].value_counts().mean()
```

```
789.6
```

✓ CLASS AUGMENTATION AND PREPROCESSING

```
# Define a function to preprocess eye fundus images
def preprocess_eye_image(image):
    if not os.path.isfile(image_path):
        raise ValueError(f"File not found: {image_path}")
    if image is None:
        raise ValueError(f"Failed to read image file: {image_path}")
    # Convert to grayscale
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Apply CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
clahe_image = clahe.apply(gray)

return clahe_image
```

```
grouped = df.groupby('class_name')
```

```
# Create a new dataframe to store the selected images
selected_df = pd.DataFrame(columns=['image_path', 'class_name'])

brightness_range = (0.5, 1.5)

# Define the augmentation pipeline
augmentation_pipeline = iaa.Sequential([
    iaa.Multiply((brightness_range[0], brightness_range[1])), # Adjust brightness
    iaa.Fliplr(p=0.5) # Horizontal flip with 50% probability
])

# Loop through each class
for class_name, group in grouped:

    augmented_folder = f'./augmented_images/{class_name}/'
    if os.path.exists(augmented_folder):
        shutil.rmtree(augmented_folder)
    destination_folder = f'./selected_images/{class_name}/'
    if os.path.exists(destination_folder):
        shutil.rmtree(destination_folder)

    # Check the number of images in the class
    n_images = len(group)

    # If the class has less than 300 images, augment the images to reach 300
    if n_images < 800:
        n_augment = 800 - n_images
        selected_images = group.sample(n=n_images, random_state=1)
        for i in range(n_augment):
            # Select a random image from the group
            index = random.choice(group.index)
            row = group.loc[index]

            # Apply augmentation to the selected image, preprocess it, and save to the new dataframe
            image = cv2.imread(row['image_path'])
            image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert to RGB
            augmented_image = augmentation_pipeline(image=image)
            augmented_image = cv2.cvtColor(augmented_image, cv2.COLOR_RGB2BGR) # Convert back to BGR

            os.makedirs(augmented_folder, exist_ok=True)
            destination_file = f'./augmented_images/{class_name}/image_aug{i}.png'
            cv2.imwrite(destination_file, augmented_image)
            selected_df = selected_df.append({'image_path': destination_file, 'class_name': class_name}, ignore_index=True)

        # Concatenate the original images and the newly augmented images
        selected_images = pd.concat([selected_images, selected_df[selected_df['class_name'] == class_name]], ignore_index=True)

    # If the class has more than 300 images, randomly select 300
    else:
        selected_images = group.sample(n=1200, random_state=1)

    for i, row in tqdm(selected_images.iterrows(), total=len(selected_images), desc=f'Preprocessing {class_name}'):
        image = cv2.imread(row['image_path'])
        print(row['image_path'])
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert to RGB

        preprocessed_image = preprocess_eye_image(image)
        os.makedirs(destination_folder, exist_ok=True)
        destination_file = f'./selected_images/{class_name}/image{i}.png'
        cv2.imwrite(destination_file, preprocessed_image)
        selected_df = selected_df.append({'image_path': destination_file, 'class_name': class_name}, ignore_index=True)
```

Preprocessing normal: 16% | 186/1200 [00:48<04:35, 3.69it/s] ./datasets/normal/image1672.png

```

Preprocessing normal: 16% | 187/1200 [00:48<04:40, 3.61it/s] ./datasets/normal/image1816.png
Preprocessing normal: 16% | 188/1200 [00:49<04:22, 3.86it/s] ./datasets/normal/image1415.png
Preprocessing normal: 16% | 189/1200 [00:49<04:12, 4.01it/s] ./datasets/normal/image2590.png
Preprocessing normal: 16% | 190/1200 [00:49<04:07, 4.08it/s] ./datasets/normal/image1796.png
Preprocessing normal: 16% | 191/1200 [00:49<04:11, 4.01it/s] ./datasets/normal/image1393.png
Preprocessing normal: 16% | 192/1200 [00:50<04:25, 3.79it/s] ./datasets/normal/image1522.png
Preprocessing normal: 16% | 193/1200 [00:50<04:12, 3.99it/s] ./datasets/normal/image927.png
Preprocessing normal: 16% | 194/1200 [00:50<03:59, 4.20it/s] ./datasets/normal/image912.png
Preprocessing normal: 16% | 195/1200 [00:50<03:58, 4.21it/s] ./datasets/normal/image2045.png
Preprocessing normal: 16% | 196/1200 [00:51<03:56, 4.25it/s] ./datasets/normal/image542.png
Preprocessing normal: 16% | 197/1200 [00:51<03:48, 4.39it/s] ./datasets/normal/image2084.png
Preprocessing normal: 16% | 198/1200 [00:51<03:48, 4.38it/s] ./datasets/normal/image1396.png
Preprocessing normal: 17% | 199/1200 [00:51<04:53, 3.41it/s] ./datasets/normal/image957.png
Preprocessing normal: 17% | 200/1200 [00:52<04:55, 3.38it/s] ./datasets/normal/image410.png
Preprocessing normal: 17% | 201/1200 [00:52<04:36, 3.62it/s] ./datasets/normal/image2576.png
Preprocessing normal: 17% | 202/1200 [00:52<04:22, 3.81it/s] ./datasets/normal/image1909.png
Preprocessing normal: 17% | 203/1200 [00:52<04:16, 3.88it/s] ./datasets/normal/image2191.png
Preprocessing normal: 17% | 204/1200 [00:53<04:02, 4.10it/s] ./datasets/normal/image779.png
Preprocessing normal: 17% | 205/1200 [00:53<03:55, 4.23it/s] ./datasets/normal/image941.png
Preprocessing normal: 17% | 206/1200 [00:53<03:48, 4.35it/s] ./datasets/normal/image871.png
Preprocessing normal: 17% | 207/1200 [00:53<03:51, 4.28it/s] ./datasets/normal/image707.png
Preprocessing normal: 17% | 208/1200 [00:54<03:46, 4.38it/s] ./datasets/normal/image972.png
Preprocessing normal: 17% | 209/1200 [00:54<04:05, 4.03it/s] ./datasets/normal/image2172.png
Preprocessing normal: 18% | 210/1200 [00:54<03:58, 4.16it/s] ./datasets/normal/image2689.png
Preprocessing normal: 18% | 211/1200 [00:54<04:07, 4.00it/s] ./datasets/normal/image2404.png
Preprocessing normal: 18% | 212/1200 [00:55<03:54, 4.21it/s] ./datasets/normal/image1844.png
Preprocessing normal: 18% | 213/1200 [00:55<03:45, 4.37it/s] ./datasets/normal/image805.png
Preprocessing normal: 18% | 214/1200 [00:55<03:52, 4.24it/s] ./datasets/normal/image236.png
./datasets/normal/image2585.png
Preprocessing normal: 18% | 216/1200 [00:55<03:38, 4.51it/s] ./datasets/normal/image2718.png
Preprocessing normal: 18% | 217/1200 [00:56<03:40, 4.46it/s] ./datasets/normal/image30.png
Preprocessing normal: 18% | 218/1200 [00:56<03:54, 4.19it/s] ./datasets/normal/image1142.png
Preprocessing normal: 18% | 219/1200 [00:56<03:49, 4.27it/s] ./datasets/normal/image994.png
Preprocessing normal: 18% | 220/1200 [00:56<04:09, 3.93it/s] ./datasets/normal/image2377.png
Preprocessing normal: 18% | 221/1200 [00:57<04:19, 3.77it/s] ./datasets/normal/image1505.png
Preprocessing normal: 18% | 222/1200 [00:57<04:26, 3.66it/s] ./datasets/normal/image979.png
Preprocessing normal: 19% | 223/1200 [00:57<04:17, 3.79it/s] ./datasets/normal/image807.png
Preprocessing normal: 19% | 224/1200 [00:58<04:05, 3.98it/s] ./datasets/normal/image1454.png
Preprocessing normal: 19% | 225/1200 [00:58<03:52, 4.20it/s] ./datasets/normal/image309.png
Preprocessing normal: 19% | 226/1200 [00:58<03:47, 4.29it/s] ./datasets/normal/image424.png
Preprocessing normal: 19% | 227/1200 [00:58<03:59, 4.07it/s] ./datasets/normal/image2001.png
Preprocessing normal: 19% | 228/1200 [00:58<03:58, 4.07it/s] ./datasets/normal/image382.png
Preprocessing normal: 19% | 229/1200 [00:59<03:50, 4.21it/s] ./datasets/normal/image988.png
Preprocessing normal: 19% | 230/1200 [00:59<04:16, 3.79it/s] ./datasets/normal/image331.png
Preprocessing normal: 19% | 231/1200 [00:59<04:28, 3.60it/s] ./datasets/normal/image2100.png
Preprocessing normal: 19% | 232/1200 [01:00<04:17, 3.76it/s] ./datasets/normal/image1991.png
Preprocessing normal: 19% | 233/1200 [01:00<04:53, 3.29it/s] ./datasets/normal/image2208.png
Preprocessing normal: 20% | 234/1200 [01:00<04:39, 3.46it/s] ./datasets/normal/image2571.png
Preprocessing normal: 20% | 235/1200 [01:00<04:17, 3.75it/s] ./datasets/normal/image1690.png
Preprocessing normal: 20% | 236/1200 [01:01<04:19, 3.71it/s] ./datasets/normal/image1342.png
Preprocessing normal: 20% | 237/1200 [01:01<04:33, 3.52it/s] ./datasets/normal/image626.png
Preprocessing normal: 20% | 238/1200 [01:01<04:34, 3.50it/s] ./datasets/normal/image1787.png

```

selected_df

	image_path	class_name
0	./augmented_images/ageDegeneration/image_aug0.png	ageDegeneration
1	./augmented_images/ageDegeneration/image_aug1.png	ageDegeneration
2	./augmented_images/ageDegeneration/image_aug2.png	ageDegeneration
3	./augmented_images/ageDegeneration/image_aug3.png	ageDegeneration
4	./augmented_images/ageDegeneration/image_aug4.png	ageDegeneration
...	...	...
6520	./selected_images/normal/image1981.png	normal
6521	./selected_images/normal/image1514.png	normal
6522	./selected_images/normal/image593.png	normal
6523	./selected_images/normal/image1084.png	normal
6524	./selected_images/normal/image1932.png	normal

6525 rows x 2 columns

final_selected_df = selected_df

```
pattern = re.compile(r'aug\d*\.\png$')
```

```
final_selected_df = final_selected_df[~final_selected_df['image_path'].str.contains(pattern)]
```

```
final_selected_df
```

	image_path	class_name
534	./selected_images/ageDegeneration/image0.png	ageDegeneration
535	./selected_images/ageDegeneration/image1.png	ageDegeneration
536	./selected_images/ageDegeneration/image2.png	ageDegeneration
537	./selected_images/ageDegeneration/image3.png	ageDegeneration
538	./selected_images/ageDegeneration/image4.png	ageDegeneration
...	...	...
6520	./selected_images/normal/image1981.png	normal
6521	./selected_images/normal/image1514.png	normal
6522	./selected_images/normal/image593.png	normal
6523	./selected_images/normal/image1084.png	normal
6524	./selected_images/normal/image1932.png	normal

4400 rows x 2 columns

```
df.to_csv('final_selected_df.csv')
```

```
def file_count(dir_path):
    count = 0
    # Iterate directory
    for path in os.listdir(dir_path):
        # check if current path is a file
        if os.path.isfile(os.path.join(dir_path, path)):
            count += 1
    print(dir_path + ' File count:', count)
```

```
for dir in os.listdir('./selected_images'):
    file_count('./selected_images/'+dir)
```

```
./selected_images/ageDegeneration File count: 800
./selected_images/cataract File count: 800
./selected_images/glaucoma File count: 800
./selected_images/myopia File count: 800
./selected_images/normal File count: 1200
```

```
from google.colab.patches import cv2_imshow

img = cv2.imread('/content/drive/MyDrive/ODR/datasets/glaucoma/image3.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
Img = preprocess_eye_image(img)
Img_rgb = cv2.cvtColor(Img, cv2.COLOR_BGR2RGB)

fig, axs = plt.subplots(1, 2, figsize=(10, 5))

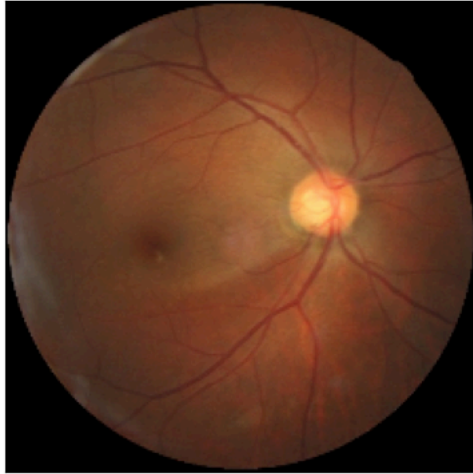
axs[0].set_title('Before Preprocessing')
axs[1].set_title('After Preprocessing')

axs[0].imshow(img)
axs[1].imshow(Img_rgb, cmap=None)

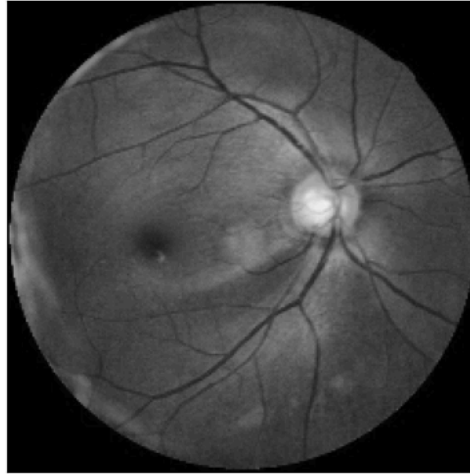
for ax in axs:
    ax.set_xticks([])
    ax.set_yticks([])

plt.show()
```

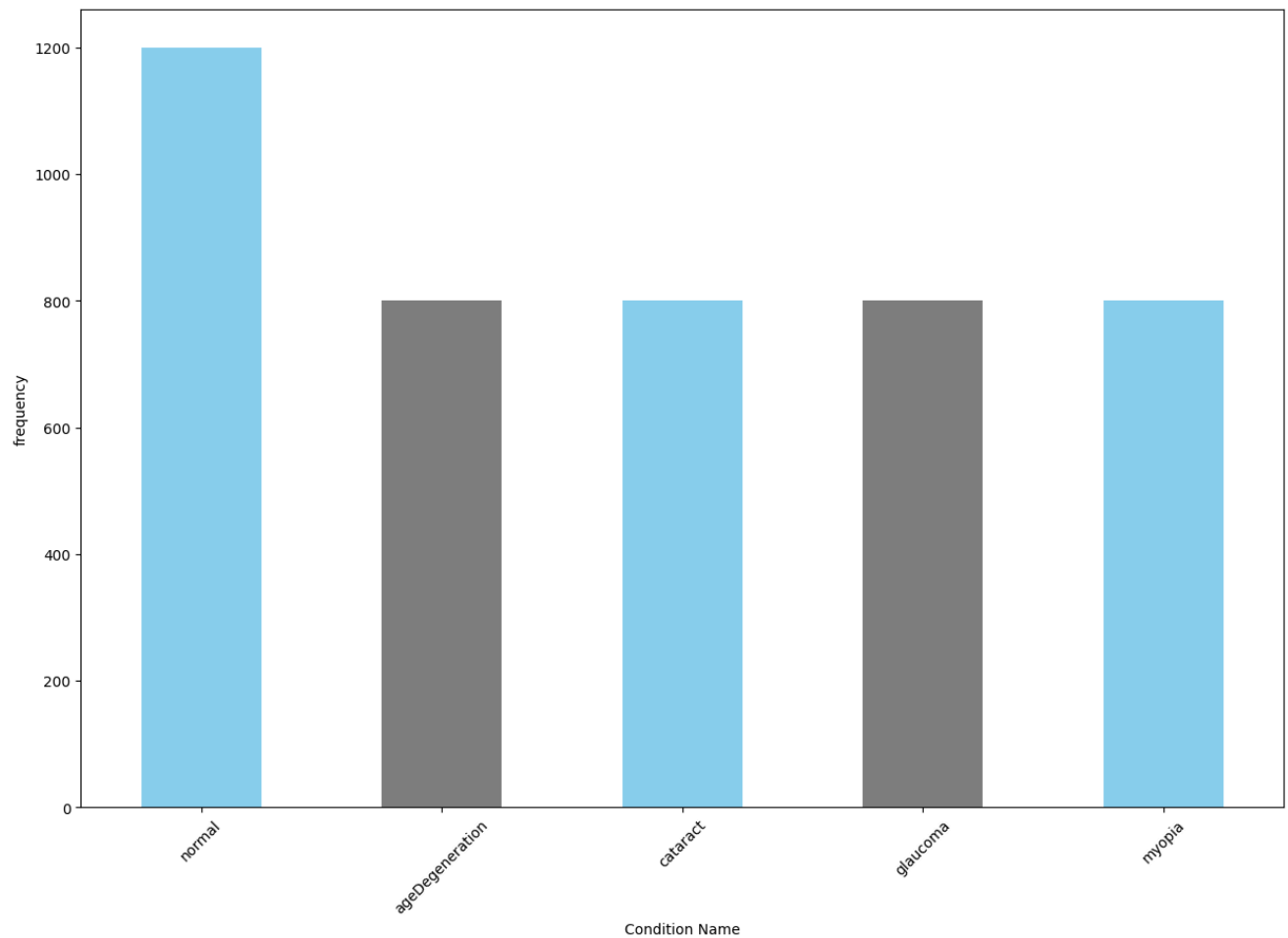
Before Preprocessing



After Preprocessing



```
fig = plt.figure(figsize =(15, 10))
final_selected_df['class_name'].value_counts().plot(kind='bar', xlabel='Condition Name', ylabel='frequency', color=[
plt.xticks(rotation=45)
plt.show()
plt.savefig('AfterSolvingClassImbalance.png')
```



<Figure size 640x480 with 0 Axes>

✓ TRAIN, VALIDATION AND TEST GENERATORS

```
base_dir = './splits'
if os.path.exists(base_dir):
    shutil.rmtree(base_dir)
os.mkdir(base_dir)
```

```
def dir_create(base_dir, name):
    curr_dir = os.path.join(base_dir, name)
    os.mkdir(curr_dir)
```

```
dir_create(base_dir, 'Training')
dir_create(base_dir, 'Testing')
dir_create(base_dir, 'Validation')

# Under train,test,validation folder create folders
# (Glaucoma, Cataract, Age related Macular Degeneration, Pathological Myopia)
for dir in ['normal', 'glaucoma', 'cataract', 'ageDegeneration', 'myopia']:
    dir_create('./splits/Training', dir)
    dir_create('./splits/Testing', dir)
    dir_create('./splits/Validation', dir)
```



```

glaucoma_source_dir = './selected_images/glaucoma/'
training_glaucoma_dir = './splits/Training/glaucoma/'
testing_glaucoma_dir = './splits/Testing/glaucoma/'
valid_glaucoma_dir = './splits/Validation/glaucoma/'

cataract_source_dir = './selected_images/cataract/'
training_cataract_dir = './splits/Training/cataract/'
testing_cataract_dir = './splits/Testing/cataract/'
valid_cataract_dir = './splits/Validation/cataract/'

ageDegeneration_source_dir = './selected_images/ageDegeneration/'
training_ageDegeneration_dir = './splits/Training/ageDegeneration/'
testing_ageDegeneration_dir = './splits/Testing/ageDegeneration/'
valid_ageDegeneration_dir = './splits/Validation/ageDegeneration/'

myopia_source_dir = './selected_images/myopia/'
training_myopia_dir = './splits/Training/myopia/'
testing_myopia_dir = './splits/Testing/myopia/'
valid_myopia_dir = './splits/Validation/myopia/'

normal_source_dir = './selected_images/normal/'
training_normal_dir = './splits/Training/normal/'
testing_normal_dir = './splits/Testing/normal/'
valid_normal_dir = './splits/Validation/normal/'

```

```

def split_data(SOURCE, TRAINING, VALIDATION, TESTING, SPLIT_SIZE):
    files = []
    for filename in os.listdir(SOURCE):
        file = SOURCE + filename
        if os.path.getsize(file) > 0:
            files.append(filename)
        else:
            print(filename + " is zero length, so ignoring.")

    SIZE = (1 - SPLIT_SIZE) / 2
    training_length = int(len(files) * SPLIT_SIZE)
    testing_length = int(len(files) * SIZE)
    valid_length = int(len(files) * SIZE)
    shuffled_set = random.sample(files, len(files))
    training_set = shuffled_set[0:training_length]
    testing_set = shuffled_set[training_length:training_length+testing_length]
    valid_set = shuffled_set[training_length+testing_length:]

    for filename in training_set:
        this_file = SOURCE + filename
        destination = TRAINING + filename
        shutil.copy(this_file, destination)

    for filename in valid_set:
        this_file = SOURCE + filename
        destination = VALIDATION + filename
        shutil.copy(this_file, destination)

    for filename in testing_set:
        this_file = SOURCE + filename
        destination = TESTING + filename
        shutil.copy(this_file, destination)

```

```
split_size = 0.80
```

```

split_data(glaucoma_source_dir, training_glaucoma_dir, testing_glaucoma_dir, valid_glaucoma_dir, split_size)
split_data(cataract_source_dir, training_cataract_dir, testing_cataract_dir, valid_cataract_dir, split_size)
split_data(ageDegeneration_source_dir, training_ageDegeneration_dir, testing_ageDegeneration_dir, valid_ageDegeneration_dir, split_size)
split_data(myopia_source_dir, training_myopia_dir, testing_myopia_dir, valid_myopia_dir, split_size)
split_data(normal_source_dir, training_normal_dir, testing_normal_dir, valid_normal_dir, split_size)

```

```

def img_p(img_height, img_width):
    train_datagen = ImageDataGenerator(rescale = 1./255., rotation_range = 30, width_shift_range = 0.1, height_shift_range = 0.1)
    train_generator = train_datagen.flow_from_directory('./splits/Training/', target_size=(img_height, img_width), class_mode='binary')
    validation_datagen = ImageDataGenerator(rescale=1./255)
    validation_generator = validation_datagen.flow_from_directory('./splits/Validation/', target_size=(img_height, img_width), class_mode='binary')
    test_datagen = ImageDataGenerator(rescale=1./255)
    test_generator = test_datagen.flow_from_directory('./splits/Testing/', target_size=(img_height, img_width), class_mode='binary')
    return train_generator, validation_generator, test_generator

```

✓ InceptionV3

```
train_generator, validation_generator, test_generator = img_p(299, 299)
```

```
Found 3520 images belonging to 5 classes.
Found 435 images belonging to 5 classes.
Found 445 images belonging to 5 classes.
```

```
class_labels_inception = list(train_generator.class_indices.keys())
```

```
class_labels_inception
```

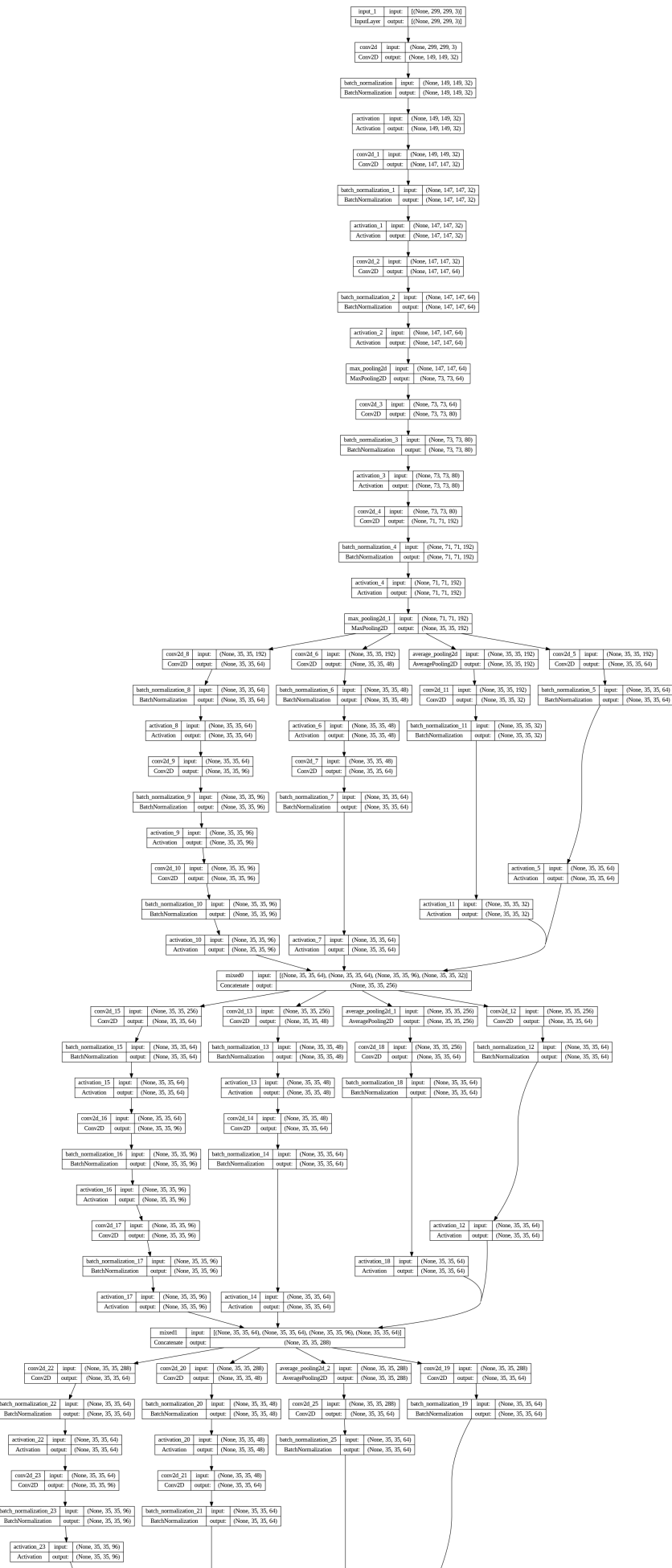
```
['ageDegeneration', 'cataract', 'glaucoma', 'myopia', 'normal']
```

```
base_model = InceptionV3(input_shape=(299, 299, 3), weights='imagenet', include_top=True)
base_model.summary()
```

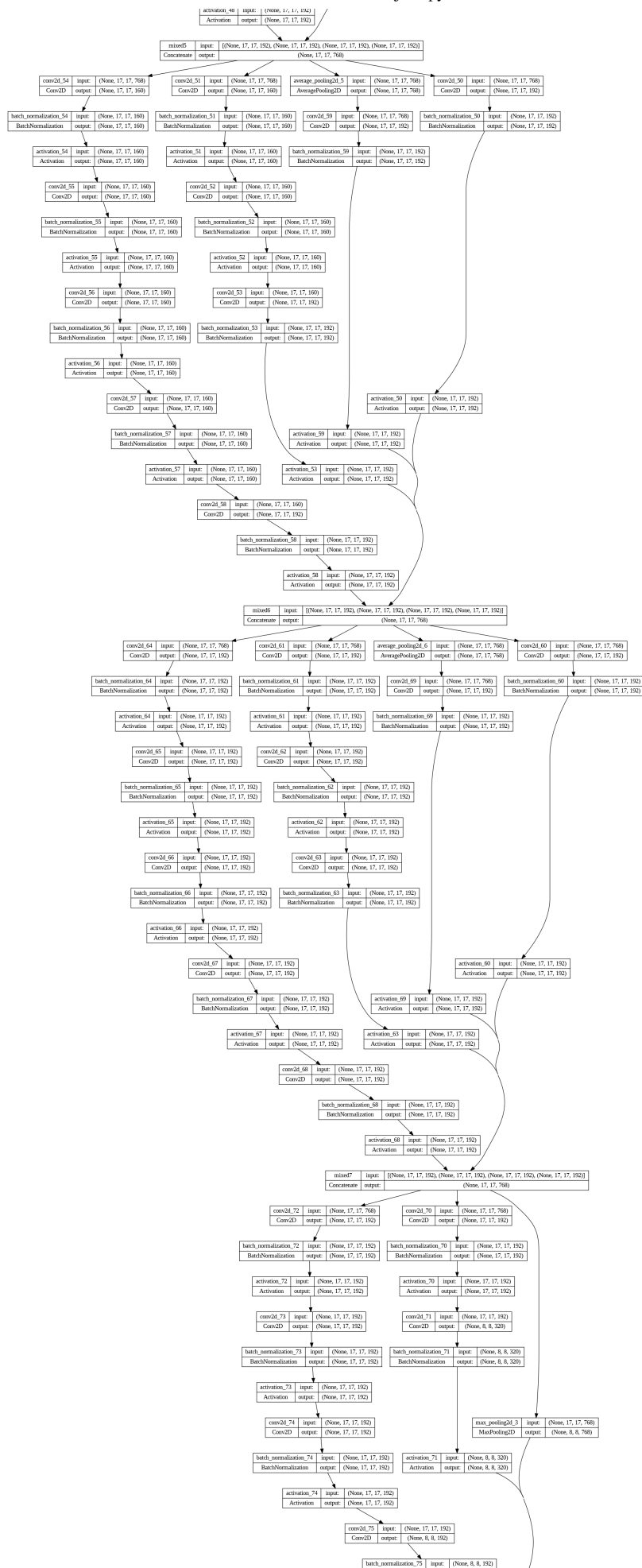
```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception\_v3/inception\_v3\_weight:
96112376/96112376 [=====] - 1s 0us/step
Model: "inception_v3"
```

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 299, 299, 3)]	0	[]
conv2d (Conv2D)	(None, 149, 149, 32)	864	['input_1[0][0]']
batch_normalization (Batch Normalization)	(None, 149, 149, 32)	96	['conv2d[0][0]']
activation (Activation)	(None, 149, 149, 32)	0	['batch_normalization[0][0]']
conv2d_1 (Conv2D)	(None, 147, 147, 32)	9216	['activation[0][0]']
batch_normalization_1 (Batch Normalization)	(None, 147, 147, 32)	96	['conv2d_1[0][0]']
activation_1 (Activation)	(None, 147, 147, 32)	0	['batch_normalization_1[0][0]']
conv2d_2 (Conv2D)	(None, 147, 147, 64)	18432	['activation_1[0][0]']
batch_normalization_2 (Batch Normalization)	(None, 147, 147, 64)	192	['conv2d_2[0][0]']
activation_2 (Activation)	(None, 147, 147, 64)	0	['batch_normalization_2[0][0]']
max_pooling2d (MaxPooling2D)	(None, 73, 73, 64)	0	['activation_2[0][0]']
conv2d_3 (Conv2D)	(None, 73, 73, 80)	5120	['max_pooling2d[0][0]']
batch_normalization_3 (Batch Normalization)	(None, 73, 73, 80)	240	['conv2d_3[0][0]']
activation_3 (Activation)	(None, 73, 73, 80)	0	['batch_normalization_3[0][0]']
conv2d_4 (Conv2D)	(None, 71, 71, 192)	138240	['activation_3[0][0]']
batch_normalization_4 (Batch Normalization)	(None, 71, 71, 192)	576	['conv2d_4[0][0]']
activation_4 (Activation)	(None, 71, 71, 192)	0	['batch_normalization_4[0][0]']
max_pooling2d_1 (MaxPooling2D)	(None, 35, 35, 192)	0	['activation_4[0][0]']
conv2d_8 (Conv2D)	(None, 35, 35, 64)	12288	['max_pooling2d_1[0][0]']
batch_normalization_8 (Batch Normalization)	(None, 35, 35, 64)	192	['conv2d_8[0][0]']

```
plot_model(base_model, to_file='inception_model.png', show_shapes=True, show_layer_names=True)
display(Image.open('inception_model.png'))
```








```
x = base_model.layers[-2].output
x = layers.Dense(1024, activation='relu')(x)
x = layers.Dense (5, activation='softmax')(x)
```

```
base_m = Model(inputs=base_model.input, outputs=x)
```

```
base_m.compile(optimizer = RMSprop(learning_rate=0.0001), loss = 'categorical_crossentropy', metrics = ['accuracy'],
```

```
early_stopping_monitor = EarlyStopping(patience=2)
```

```
clf=base_m.fit(train_generator,
               validation_data = validation_generator,
               batch_size=32,
               steps_per_epoch=110,
               epochs =10,
               validation_steps=14,
               verbose = 1,
               callbacks=[early_stopping_monitor])
```

```
Epoch 1/10
110/110 [=====] - 155s 956ms/step - loss: 0.6921 - accuracy: 0.7372 - recall: 0.6722 - preci
Epoch 2/10
110/110 [=====] - 96s 873ms/step - loss: 0.4315 - accuracy: 0.8480 - recall: 0.8259 - precis
Epoch 3/10
110/110 [=====] - 97s 880ms/step - loss: 0.3090 - accuracy: 0.8889 - recall: 0.8730 - precis
Epoch 4/10
110/110 [=====] - 96s 870ms/step - loss: 0.2508 - accuracy: 0.9116 - recall: 0.9031 - precis
Epoch 5/10
110/110 [=====] - 98s 889ms/step - loss: 0.1958 - accuracy: 0.9267 - recall: 0.9202 - precis
Epoch 6/10
110/110 [=====] - 96s 871ms/step - loss: 0.1722 - accuracy: 0.9432 - recall: 0.9389 - precis
```

```
base_m.evaluate(test_generator)
```

```
14/14 [=====] - 5s 326ms/step - loss: 0.8093 - accuracy: 0.8809 - recall: 0.8809 - precision
[0.8093124032020569,
 0.8808988928794861,
 0.8808988928794861,
 0.8808988928794861,
 0.9663318991661072,
 0.8883414268493652]
```

```
acc = clf.history['accuracy']
val_acc = clf.history['val_accuracy']
print(val_acc)
loss = clf.history['loss']
val_loss = clf.history['val_loss']

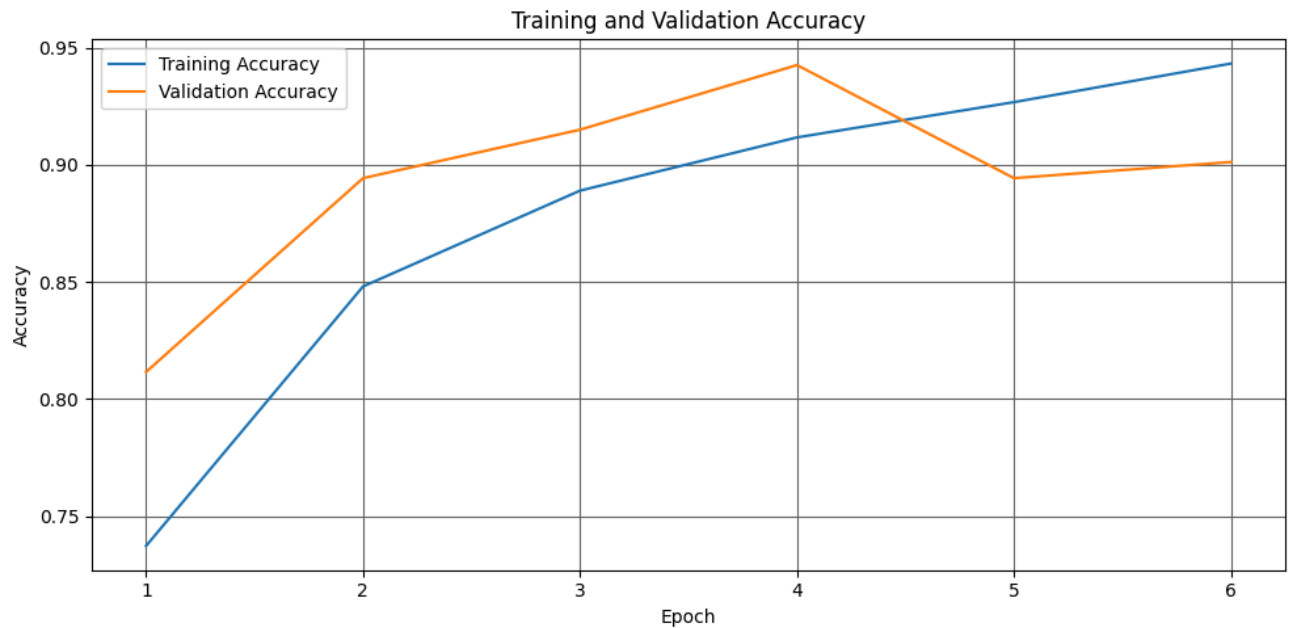
epochs = range(1, len(loss) + 1)

#accuracy plot
plt.figure(figsize=(10,5))
plt.plot(epochs, acc, label='Training Accuracy') #color='lightblue',
plt.plot(epochs, val_acc, label='Validation Accuracy') #color='orange',
plt.title('Training and Validation Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
#plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.savefig('inception_accuracy.png')

#loss plot
plt.figure(figsize=(10,5))
plt.plot(epochs, loss, color='pink', label='Training Loss')
plt.plot(epochs, val_loss, color='red', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
#plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
```

```
plt.tight_layout()
plt.rcParams['axes.facecolor'] = 'white'
plt.savefig('inception_loss.png')
```

[0.8114942312240601, 0.8942528963088989, 0.9149425029754639, 0.9425287246704102, 0.8942528963088989, 0.9011494517326]



```
acc = clf.history['accuracy']
val_acc = clf.history['val_accuracy']
print(val_acc)
loss = clf.history['loss']
val_loss = clf.history['val_loss']

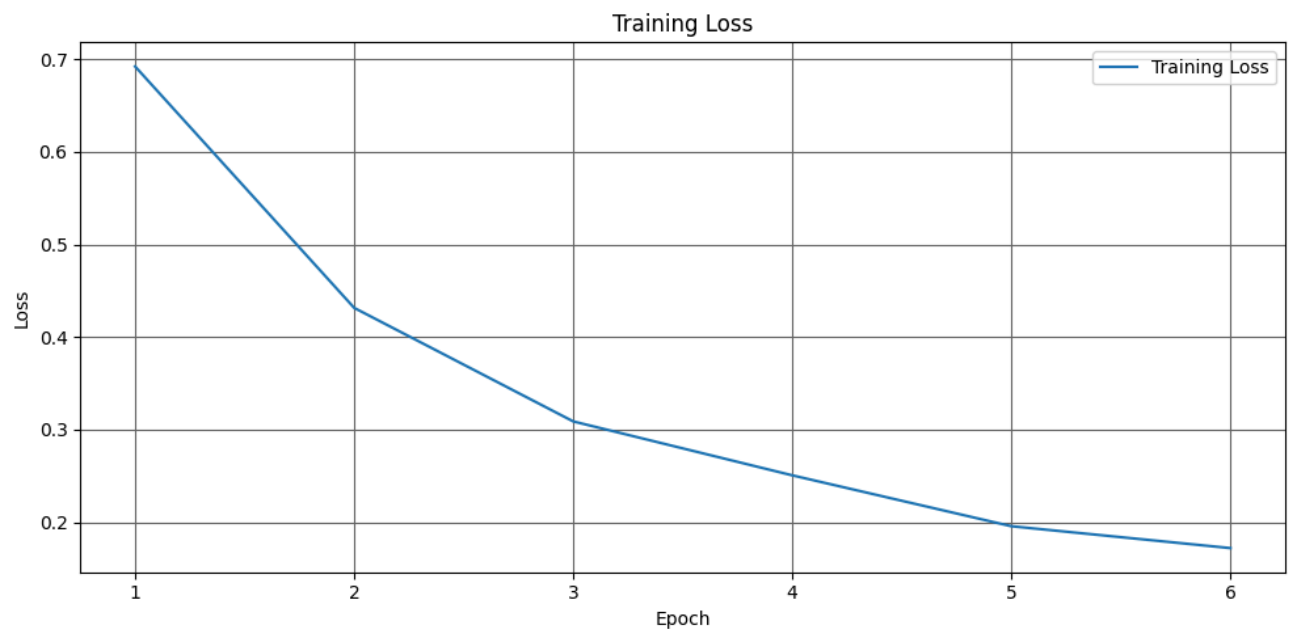
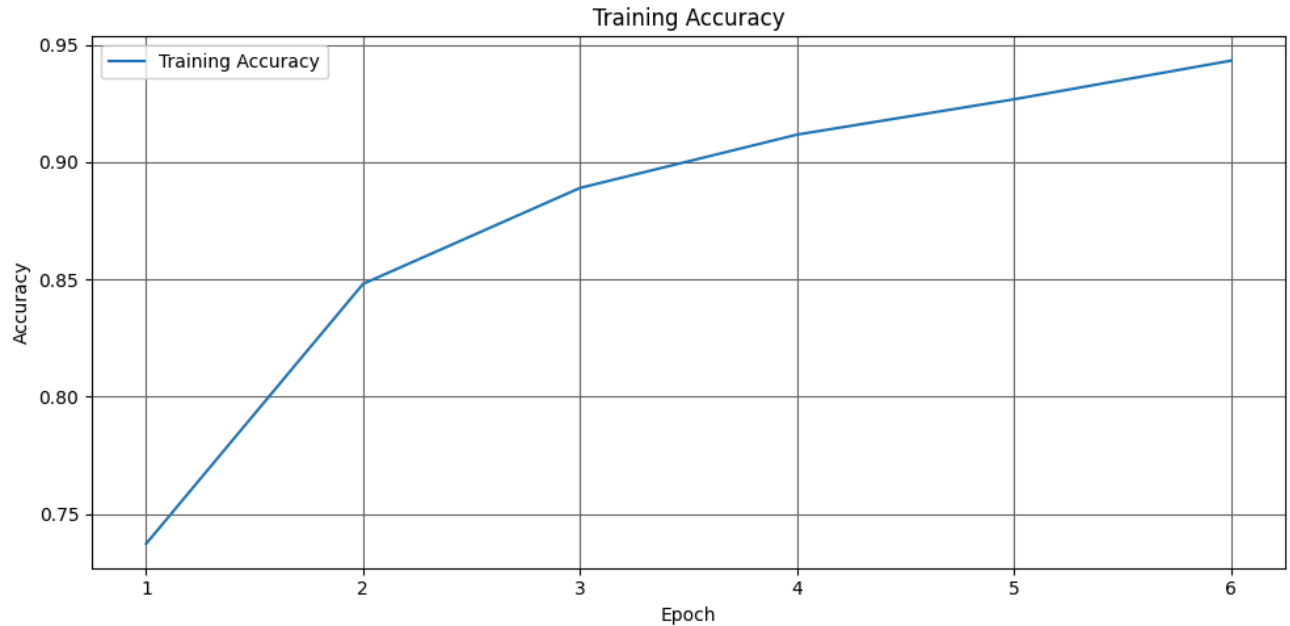
epochs = range(1, len(loss) + 1)

#accuracy plot
plt.figure(figsize=(10,5))
plt.plot(epochs, acc, label='Training Accuracy') #color='lightblue',
#plt.plot(epochs, val_acc, label='Validation Accuracy') #color='orange',
plt.title('Training Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
#plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
```

```
plt.tight_layout()
plt.savefig('inception_accuracy.png')

#loss plot
plt.figure(figsize=(10,5))
plt.plot(epochs, loss, label='Training Loss')
#plt.plot(epochs, val_loss, color='red', label='Validation Loss')
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
#plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.rcParams['axes.facecolor'] = 'white'
plt.savefig('inception_loss.png')
```

[0.8114942312240601, 0.8942528963088989, 0.9149425029754639, 0.9425287246704102, 0.8942528963088989, 0.9011494517321]



```
base_m.save("inception_v3_800.h5")
```

```
inception_pred = base_m.predict(test_generator)
```

14/14 [=====] - 3s 211ms/step

▼ reserve

```
m = Model(inputs=base_model.input, outputs=base_model.layers[-2].output)
```

```
features = m.predict(train_generator)
```

```
30/30 [=====] - 23s 737ms/step
```

```
features.shape
```

```
(960, 2048)
```

```
t_features = m.predict(test_generator)
```

```
4/4 [=====] - 1s 132ms/step
```

```
t_features.shape
```

```
(124, 2048)
```

```
from numpy import save
save('train_fe', features)
save('test_fe', t_features)
```

```
local_weights_file = '/content/drive/MyDrive/ODR/inception_v3_weights_tf_dim_ordering_tf_kernels_notop.h5'
pre_trained_model = InceptionV3(input_shape = (299, 299, 3),
                                include_top = False,
                                weights = None)
pre_trained_model.load_weights(local_weights_file)
for layer in pre_trained_model.layers:
    layer.trainable = False
last_layer = pre_trained_model.get_layer('mixed7')
print('last layer output shape: ', last_layer.output_shape)
last_output = last_layer.output
```

```
last layer output shape: (None, 17, 17, 768)
```

```
x = layers.Flatten()(last_output)
x = layers.Dropout(0.5)(x)
x = layers.Dense(1024, activation='relu')(x)
x = layers.Dense(4, activation='softmax')(x)
model = Model(pre_trained_model.input, x)
model.compile(optimizer = RMSprop(learning_rate=0.0001),
              loss = 'categorical_crossentropy',
              metrics = ['accuracy', tf.keras.metrics.Recall(), tf.keras.metrics.Precision(), tf.keras.metrics.AUC()])
```

```
model.summary()
```

```
Model: "model_2"
```

Layer (type)	Output Shape	Param #	Connected to
=====			
input_2 (InputLayer)	[(None, 299, 299, 3)]	0	[]
conv2d_94 (Conv2D)	(None, 149, 149, 32)	864	['input_2[0][0]']
batch_normalization_94 (Batch Normalization)	(None, 149, 149, 32)	96	['conv2d_94[0][0]']
activation_94 (Activation)	(None, 149, 149, 32)	0	['batch_normalization_94[0][0]']
conv2d_95 (Conv2D)	(None, 147, 147, 32)	9216	['activation_94[0][0]']
batch_normalization_95 (Batch Normalization)	(None, 147, 147, 32)	96	['conv2d_95[0][0]']
activation_95 (Activation)	(None, 147, 147, 32)	0	['batch_normalization_95[0][0]']

```

conv2d_96 (Conv2D)          (None, 147, 147, 64  18432      ['activation_95[0][0]']
)

batch_normalization_96 (BatchN (None, 147, 147, 64  192      ['conv2d_96[0][0]']
ormalization)              )

activation_96 (Activation)     (None, 147, 147, 64  0        ['batch_normalization_96[0][0]']
)

max_pooling2d_4 (MaxPooling2D) (None, 73, 73, 64)  0        ['activation_96[0][0]']

conv2d_97 (Conv2D)          (None, 73, 73, 80)   5120     ['max_pooling2d_4[0][0]']

batch_normalization_97 (BatchN (None, 73, 73, 80)  240      ['conv2d_97[0][0]']
ormalization)

activation_97 (Activation)     (None, 73, 73, 80)   0        ['batch_normalization_97[0][0]']

conv2d_98 (Conv2D)          (None, 71, 71, 192)  138240    ['activation_97[0][0]']

batch_normalization_98 (BatchN (None, 71, 71, 192)  576      ['conv2d_98[0][0]']
ormalization)

activation_98 (Activation)     (None, 71, 71, 192)  0        ['batch_normalization_98[0][0]']

max_pooling2d_5 (MaxPooling2D) (None, 35, 35, 192)  0        ['activation_98[0][0]']

conv2d_102 (Conv2D)          (None, 35, 35, 64)   12288     ['max_pooling2d_5[0][0]']

batch_normalization_102 (Batch (None, 35, 35, 64)  192      ['conv2d_102[0][0]']
Normalization)

activation_102 (Activation)     (None, 35, 35, 64)   0        ['batch_normalization_102[0][0]']

```

```

history = model.fit(
    train_generator,
    validation_data = validation_generator,
    batch_size=32,
    steps_per_epoch=30,
    epochs =10,
    validation_steps=4,
    verbose = 1,
    callbacks=[early_stopping_monitor])

```

```

Epoch 1/10
30/30 [=====] - 30s 822ms/step - loss: 2.5450 - accuracy: 0.5219 - recall_1: 0.4521 - precis
Epoch 2/10
30/30 [=====] - 24s 801ms/step - loss: 0.8118 - accuracy: 0.6885 - recall_1: 0.6385 - precis
Epoch 3/10
30/30 [=====] - 25s 828ms/step - loss: 0.6392 - accuracy: 0.7479 - recall_1: 0.7000 - precis
Epoch 4/10
30/30 [=====] - 24s 806ms/step - loss: 0.5932 - accuracy: 0.7677 - recall_1: 0.7417 - precis
Epoch 5/10
30/30 [=====] - 25s 817ms/step - loss: 0.5457 - accuracy: 0.7760 - recall_1: 0.7521 - precis
Epoch 6/10
30/30 [=====] - 24s 815ms/step - loss: 0.4848 - accuracy: 0.8198 - recall_1: 0.7948 - precis
Epoch 7/10
30/30 [=====] - 24s 800ms/step - loss: 0.4454 - accuracy: 0.8260 - recall_1: 0.8104 - precis

```

```
model.evaluate(test_generator)
```

```

4/4 [=====] - 1s 146ms/step - loss: 0.5868 - accuracy: 0.7419 - recall_1: 0.7177 - precision
[0.5867772102355957,
0.7419354915618896,
0.7177419066429138,
0.7946428656578064,
0.9425512552261353,
0.7284548282623291]

```

```
model.save("inception_v3_300.h5")
```

```

#plotting training values
import matplotlib.pyplot as plt

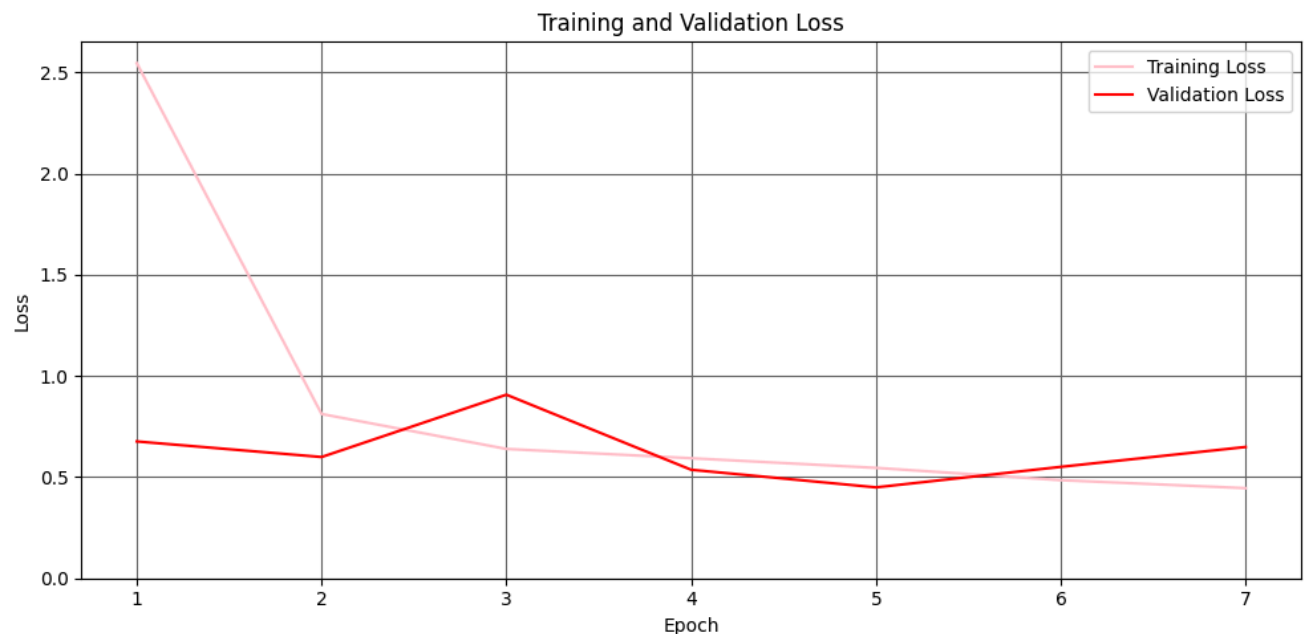
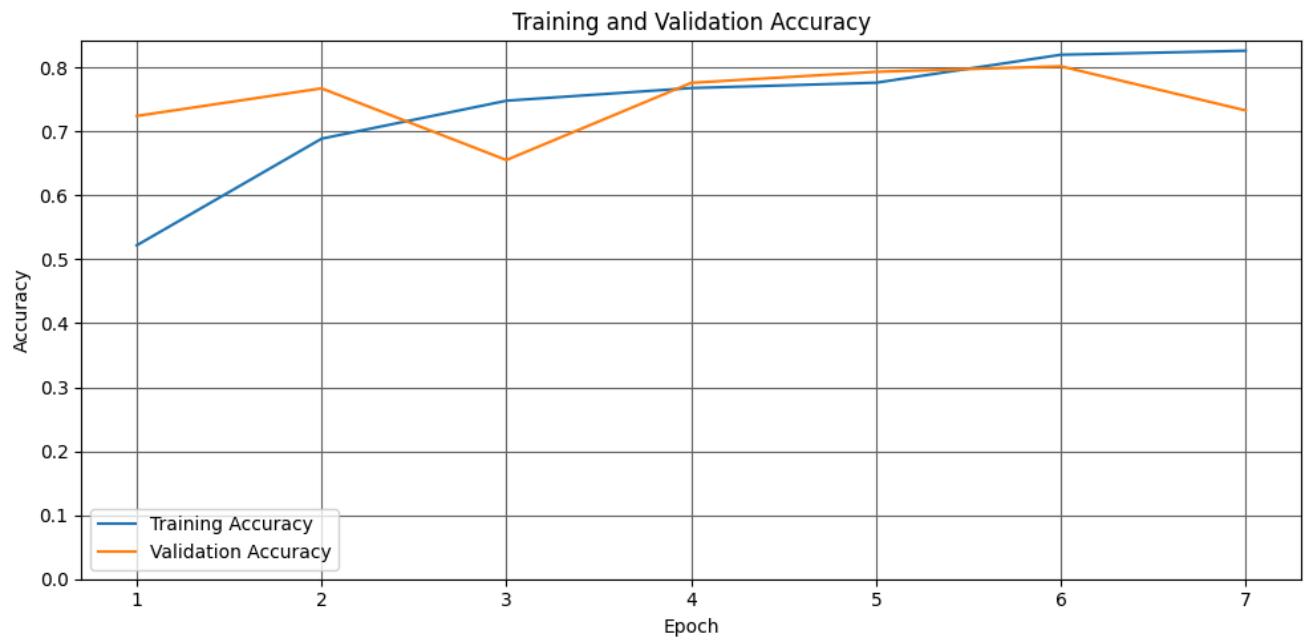
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

```

```
epochs = range(1, len(loss) + 1)

#accuracy plot
plt.figure(figsize=(10,5))
plt.plot(epochs, acc, label='Training Accuracy') #color='lightblue',
plt.plot(epochs, val_acc, label='Validation Accuracy') #color='orange',
plt.title('Training and Validation Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.savefig('inception_weights_accuracy.png')

#loss plot
plt.figure(figsize=(10,5))
plt.plot(epochs, loss, color='pink', label='Training Loss')
plt.plot(epochs, val_loss, color='red', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.rcParams['axes.facecolor'] = 'white'
plt.savefig('inception_weights_loss.png')
```



```
Y_pred1 = model.predict_generator(validation_generator, 116 // 32+1)
y_pred1 = np.argmax(Y_pred, axis=1)
print('Confusion Matrix')
print(confusion_matrix(validation_generator.classes, y_pred))
print('Classification Report')
target_names = ['galucoma', 'cataract', 'ageDegeneration', 'myopia']
print(classification_report(validation_generator.classes, y_pred, target_names=target_names))
```

<ipython-input-56-5a911a2af765>:1: UserWarning: `Model.predict\_generator` is deprecated and will be removed in a future version.

Y_pred1 = model.predict_generator(validation_generator, 116 // 32+1)

Confusion Matrix

```
[[ 7  4 15  3]
 [ 7  4  7 11]
 [ 7  8  9  5]
 [ 4 10  9  6]]
```

Classification Report

	precision	recall	f1-score	support
galucoma	0.28	0.24	0.26	29
cataract	0.15	0.14	0.15	29
ageDegeneration	0.23	0.31	0.26	29
myopia	0.24	0.21	0.22	29
accuracy			0.22	116
macro avg	0.22	0.22	0.22	116

weighted avg	0.22	0.22	0.22	116
--------------	------	------	------	-----

ResNet

```
class_labels_resnet = list(train_generator.class_indices.keys())
```

```
class_labels_resnet
```

```
['ageDegeneration', 'cataract', 'glaucoma', 'myopia', 'normal']
```

```
resnet_m = ResNet50(input_shape=(224, 224, 3), weights='imagenet', include_top=True)
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_and_1024_channels_incl_top.h5 [=====] - 1s 0us/step

```
resnet_m.summary()
```

Model: "resnet50"

Layer (type)	Output Shape	Param #	Connected to
input_2 (InputLayer)	(None, 224, 224, 3)	0	input_2 [0] [0]'
conv1_pad (ZeroPadding2D)	(None, 230, 230, 3)	0	input_2 [0] [0]'
conv1_conv (Conv2D)	(None, 112, 112, 64)	9472	conv1_pad [0] [0]'
conv1_bn (BatchNormalization)	(None, 112, 112, 64)	256	conv1_conv [0] [0]'
conv1_relu (Activation)	(None, 112, 112, 64)	0	conv1_bn [0] [0]'
pool1_pad (ZeroPadding2D)	(None, 114, 114, 64)	0	conv1_relu [0] [0]'
pool1_pool (MaxPooling2D)	(None, 56, 56, 64)	0	pool1_pad [0] [0]'
conv2_block1_1_conv (Conv2D)	(None, 56, 56, 64)	4160	pool1_pool [0] [0]'
conv2_block1_1_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block1_1_conv [0] [0]'
conv2_block1_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_1_bn [0] [0]'
conv2_block1_2_conv (Conv2D)	(None, 56, 56, 64)	36928	conv2_block1_1_relu [0] [0]'
conv2_block1_2_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block1_2_conv [0] [0]'
conv2_block1_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_2_bn [0] [0]'
conv2_block1_0_conv (Conv2D)	(None, 56, 56, 256)	16640	pool1_pool [0] [0]'
conv2_block1_3_conv (Conv2D)	(None, 56, 56, 256)	16640	conv2_block1_2_relu [0] [0]'
conv2_block1_0_bn (BatchNormalization)	(None, 56, 56, 256)	1024	conv2_block1_0_conv [0] [0]'
conv2_block1_3_bn (BatchNormalization)	(None, 56, 56, 256)	1024	conv2_block1_3_conv [0] [0]'
conv2_block1_add (Add)	(None, 56, 56, 256)	0	conv2_block1_0_bn [0] [0]'
conv2_block1_out (Activation)	(None, 56, 56, 256)	0	conv2_block1_add [0] [0]'
conv2_block2_1_conv (Conv2D)	(None, 56, 56, 64)	16448	conv2_block1_out [0] [0]'
conv2_block2_1_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block2_1_conv [0] [0]'

```

x = resnet_m.layers[-2].output

# Add dropout with a rate of 0.5
x = layers.Dropout(0.5)(x)

# Add a dense layer with 1024 units, ReLU activation, and L2 regularization with a factor of 0.01
x = layers.Dense(1024, activation='relu', kernel_regularizer=regularizers.l2(0.01))(x)

# Add dropout with a rate of 0.5
x = layers.Dropout(0.5)(x)

# Add a dense layer with 5 units, softmax activation, and L2 regularization with a factor of 0.01
x = layers.Dense(5, activation='softmax', kernel_regularizer=regularizers.l2(0.01))(x)

```

```

from tensorflow.keras.optimizers import Adam

```

```

resnet_m = Model(inputs=resnet_m.input, outputs=x)
resnet_m.compile(optimizer = Adam(learning_rate=0.0001),
                 loss = 'categorical_crossentropy',
                 metrics = ['accuracy', tf.keras.metrics.Recall(), tf.keras.metrics.Precision(), tf.keras.metrics.AUC()])

```

```

res_clf = resnet_m.fit(train_generator,
                      batch_size = 32,
                      validation_data = validation_generator,
                      steps_per_epoch=110,
                      epochs =10,
                      validation_steps=14,
                      verbose = 1,
                      #callbacks=[early_stopping_monitor]
                      )

```

```

Epoch 1/10
58/110 [=====>.....] - ETA: 50s - loss: 16.2264 - accuracy: 0.7958 - recall_9: 0.7139 - precision
-----

```

KeyboardInterrupt Traceback (most recent call last)

<ipython-input-124-aa5de7e815f2> in <cell line: 1>()

```

----> 1 res_clf = resnet_m.fit(train_generator,
2         batch_size = 32,
3         validation_data = validation_generator,
4         steps_per_epoch=110,
5         epochs =10,

```

13 frames

```

/usr/local/lib/python3.9/dist-packages/tensorflow/python/framework/ops.py in _numpy(self)
1124 def _numpy(self):
1125     try:
-> 1126         return self._numpy_internal()
1127     except core._NotOkStatusException as e: # pylint: disable=protected-access
1128         raise core._status_to_exception(e) from None # pylint: disable=protected-access

```

KeyboardInterrupt:

```

#plotting training values
import matplotlib.pyplot as plt

acc = res_clf.history['accuracy']
val_acc = res_clf.history['val_accuracy']
loss = res_clf.history['loss']
val_loss = res_clf.history['val_loss']

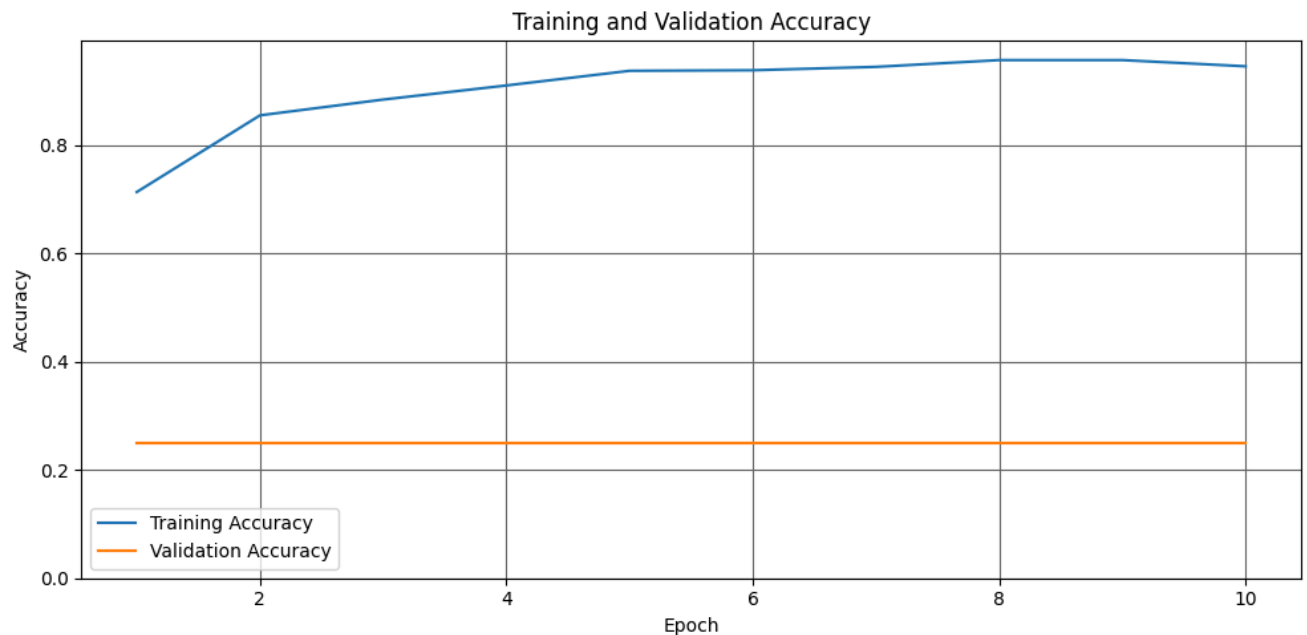
epochs = range(1, len(loss) + 1)

#accuracy plot
plt.figure(figsize=(10,5))
plt.plot(epochs, acc, label='Training Accuracy') #color='lightblue',
plt.plot(epochs, val_acc, label='Validation Accuracy') #color='orange',
plt.title('Training and Validation Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.ylim(ymin=0)
plt.legend(loc='best')
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()

```

```
plt.savefig('resnet_accuracy.png')
```

```
#loss plot
plt.figure(figsize=(10,5))
plt.plot(epochs, loss, color='pink', label='Training Loss')
plt.plot(epochs, val_loss, color='red', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.rcParams['axes.facecolor'] = 'white'
plt.savefig('resnet_loss.png')
```



```
pre_trained_model = ResNet50(input_shape = (224,224, 3),
                              include_top = False,
                              weights = 'imagenet')

for layer in pre_trained_model.layers:
    layer.trainable = False
```

```
#for layer in pre_trained_model.layers[:-10]:
    #layer.trainable = True
```

```
resNet_model = Sequential([
    pre_trained_model,
    Flatten(),
    Dropout(0.5),
    Dense(1024, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
    Dense(512, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
    Dense(5, activation='softmax')])
```

```
from tensorflow.keras.optimizers import SGD
```

```
resNet_model.compile(optimizer = SGD(learning_rate=0.0001, momentum=0.9),
    loss = 'categorical_crossentropy',
    metrics = ['accuracy', tf.keras.metrics.Recall(), tf.keras.metrics.Precision(), tf.keras.metrics.AUC()])
```

```
input_shape = (None, 224, 224, 3)
resNet_model.build(input_shape)
```

```
resNet_model.summary()
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
resnet50 (Functional)	(None, 7, 7, 2048)	23587712
flatten_6 (Flatten)	(None, 100352)	0
dropout_8 (Dropout)	(None, 100352)	0
dense_32 (Dense)	(None, 1024)	102761472
dense_33 (Dense)	(None, 512)	524800
dense_34 (Dense)	(None, 5)	2565

```
=====
Total params: 126,876,549
Trainable params: 103,288,837
Non-trainable params: 23,587,712
=====
```

```
res_clf_p = resNet_model.fit(
    train_generator,
    validation_data = validation_generator,
    steps_per_epoch=110,
    epochs =30,
    validation_steps=14,
    verbose = 1
    #callbacks=[early_stopping_monitor]
)
```

```
Epoch 1/30
110/110 [=====] - 56s 463ms/step - loss: 28.8110 - accuracy: 0.2222 - recall_16: 0.0318 - pr
Epoch 2/30
110/110 [=====] - 51s 457ms/step - loss: 28.5642 - accuracy: 0.2509 - recall_16: 0.0074 - pr
Epoch 3/30
39/110 [=====>.....] - ETA: 31s - loss: 28.4619 - accuracy: 0.2500 - recall_16: 0.0072 - precisio
```

```
KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-160-1812e9f969b5> in <cell line: 1>()
```

```
----> 1 res_clf_p = resNet_model.fit(
      2     train_generator,
      3     validation_data = validation_generator,
      4     steps_per_epoch=110,
      5     epochs =30,
```

⏮ 8 frames

```
/usr/local/lib/python3.9/dist-packages/tensorflow/python/eager/execute.py in quick_execute(op_name, num_outputs,
inputs, attrs, ctx, name)
```

```
    50 try:
    51     ctx.ensure_initialized()
----> 52     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
    53                                           inputs, attrs, num_outputs)
    54 except core._NotOkStatusException as e:
```

KeyboardInterrupt:

```
resNet_model.evaluate(test_generator)
```

```
14/14 [=====] - 3s 201ms/step - loss: 1.1125 - accuracy: 0.5191 - recall_3: 0.3663 - precisi
[1.1125054359436035,
 0.5191011428833008,
 0.36629214882850647,
 0.6735537052154541,
 0.8351855874061584,
 0.44515782594680786]
```

```
resnet_pred = resNet_model.predict(test_generator)
```

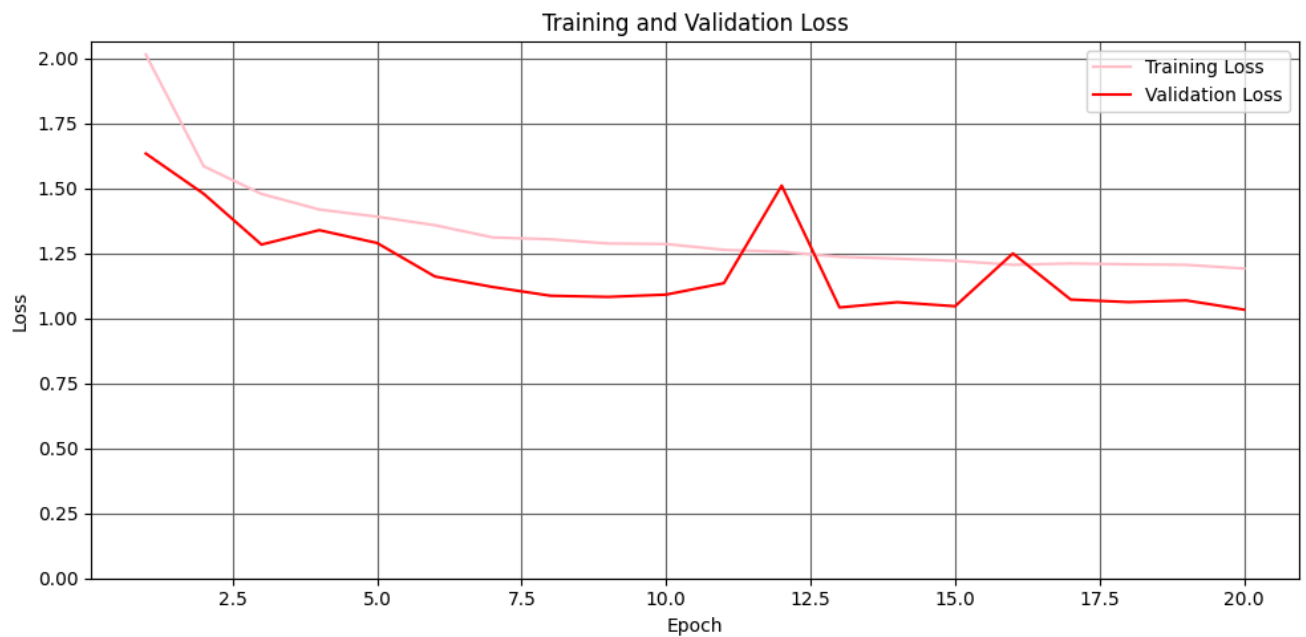
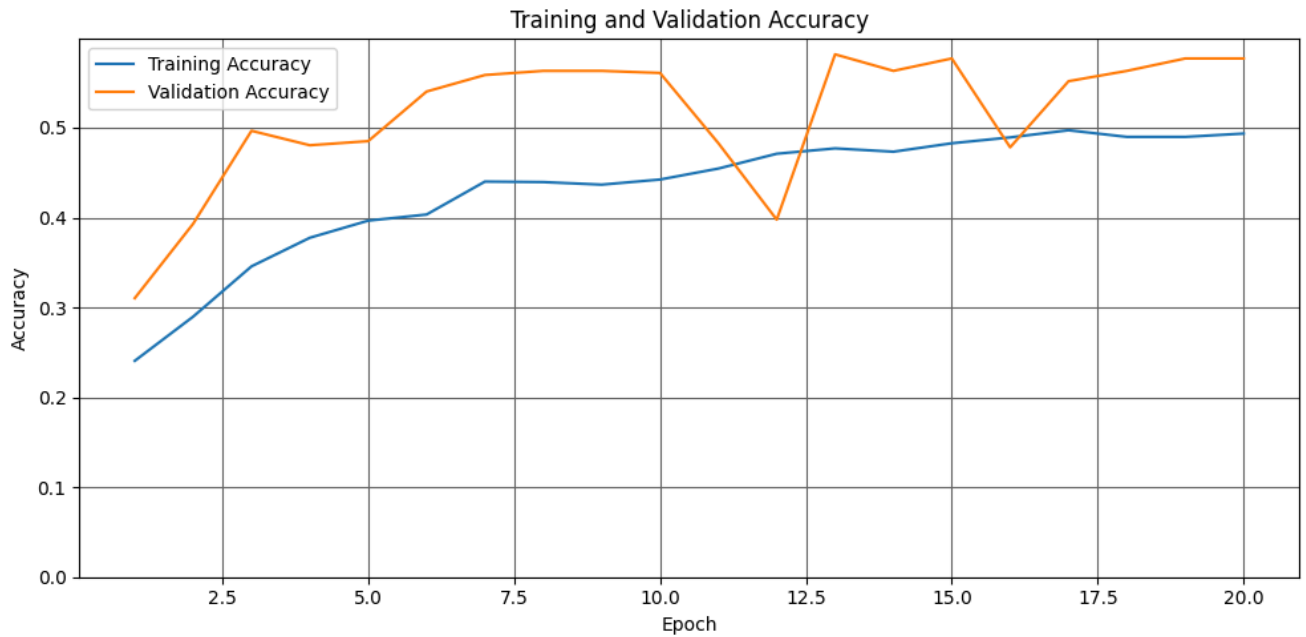
```
#plotting training values
import matplotlib.pyplot as plt
```

```
acc = res_clf_p.history['accuracy']
val_acc = res_clf_p.history['val_accuracy']
loss = res_clf_p.history['loss']
val_loss = res_clf_p.history['val_loss']
```

```
epochs = range(1, len(loss) + 1)
```

```
#accuracy plot
plt.figure(figsize=(10,5))
plt.plot(epochs, acc, label='Training Accuracy') #color='lightblue',
plt.plot(epochs, val_acc, label='Validation Accuracy') #color='orange',
plt.title('Training and Validation Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.savefig('resnet_weights_accuracy.png')
```

```
#loss plot
plt.figure(figsize=(10,5))
plt.plot(epochs, loss, color='pink', label='Training Loss')
plt.plot(epochs, val_loss, color='red', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.rcParams['axes.facecolor'] = 'white'
plt.savefig('resnet_weights_loss.png')
```



```
resNet_model.save("resnet_800.h5")
```

✓ VGG16

```
train_generator, validation_generator, test_generator = img_p(224, 224)
```

```
Found 3520 images belonging to 5 classes.
Found 435 images belonging to 5 classes.
Found 445 images belonging to 5 classes.
```

```
model=vgg16.VGG16(weights='imagenet', input_shape=(224, 224, 3), include_top=True, pooling="avg")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16\_weights\_tf\_dim\_ordering\_553467096/553467096 [=====] - 3s 0us/step
```

```
model.summary()
```

```
Model: "vgg16"
```

Layer (type)	Output Shape	Param #
--------------	--------------	---------

```

=====
input_4 (InputLayer)      [(None, 224, 224, 3)]    0
block1_conv1 (Conv2D)     (None, 224, 224, 64)     1792
block1_conv2 (Conv2D)     (None, 224, 224, 64)     36928
block1_pool (MaxPooling2D) (None, 112, 112, 64)    0
block2_conv1 (Conv2D)     (None, 112, 112, 128)    73856
block2_conv2 (Conv2D)     (None, 112, 112, 128)    147584
block2_pool (MaxPooling2D) (None, 56, 56, 128)     0
block3_conv1 (Conv2D)     (None, 56, 56, 256)     295168
block3_conv2 (Conv2D)     (None, 56, 56, 256)     590080
block3_conv3 (Conv2D)     (None, 56, 56, 256)     590080
block3_pool (MaxPooling2D) (None, 28, 28, 256)     0
block4_conv1 (Conv2D)     (None, 28, 28, 512)     1180160
block4_conv2 (Conv2D)     (None, 28, 28, 512)     2359808
block4_conv3 (Conv2D)     (None, 28, 28, 512)     2359808
block4_pool (MaxPooling2D) (None, 14, 14, 512)     0
block5_conv1 (Conv2D)     (None, 14, 14, 512)     2359808
block5_conv2 (Conv2D)     (None, 14, 14, 512)     2359808
block5_conv3 (Conv2D)     (None, 14, 14, 512)     2359808
block5_pool (MaxPooling2D) (None, 7, 7, 512)       0
flatten (Flatten)         (None, 25088)            0
fc1 (Dense)               (None, 4096)             102764544
fc2 (Dense)               (None, 4096)             16781312
predictions (Dense)       (None, 1000)             4097000
=====
Total params: 138,357,544
Trainable params: 138,357,544
Non-trainable params: 0

```

```

x = model.layers[-2].output
x = layers.Dense(1024, activation='relu')(x)
x = layers.Dense (5, activation='softmax')(x)

```

```

b_mvgg16 = Model(inputs=model.input, outputs=x)
b_mvgg16.compile(optimizer = RMSprop(learning_rate=0.0003),
                 loss = 'categorical_crossentropy',
                 metrics = ['accuracy', tf.keras.metrics.Recall(), tf.keras.metrics.Precision(), tf.keras.metrics.AUC()],

```

```

vgg_clf = b_mvgg16.fit(train_generator,
                       validation_data = validation_generator,
                       steps_per_epoch=110,
                       epochs =10,
                       validation_steps=14,
                       verbose = 1,
                       callbacks = [early_stopping_monitor]
)

```

```

Epoch 1/10
110/110 [=====] - 65s 550ms/step - loss: 0.6490 - accuracy: 0.7838 - recall_7: 0.7384 - prec
Epoch 2/10
110/110 [=====] - 61s 555ms/step - loss: 0.5819 - accuracy: 0.7980 - recall_7: 0.7588 - prec
Epoch 3/10
110/110 [=====] - 62s 556ms/step - loss: 0.5196 - accuracy: 0.8227 - recall_7: 0.7886 - prec
Epoch 4/10
110/110 [=====] - 62s 557ms/step - loss: 0.5269 - accuracy: 0.8196 - recall_7: 0.7903 - prec
Epoch 5/10

```

```
110/110 [=====] - 61s 557ms/step - loss: 0.5483 - accuracy: 0.8202 - recall_7: 0.7920 - prec
Epoch 6/10
110/110 [=====] - 61s 555ms/step - loss: 0.4925 - accuracy: 0.8335 - recall_7: 0.8062 - prec
```

```
b_mvgg16.evaluate(test_generator)
```

```
14/14 [=====] - 2s 127ms/step - loss: 0.4531 - accuracy: 0.8360 - recall_7: 0.8000 - precisi
[0.45308083295822144,
 0.8359550833702087,
 0.800000011920929,
 0.861985445022583,
 0.9737867116928101,
 0.8449104428291321]
```

```
b_mvgg16.save("vgg_800.h5")
```

```
#plotting training values
import matplotlib.pyplot as plt

acc = vgg_clf.history['accuracy']
val_acc = vgg_clf.history['val_accuracy']
loss = vgg_clf.history['loss']
val_loss = vgg_clf.history['val_loss']

epochs = range(1, len(loss) + 1)

#accuracy plot
plt.figure(figsize=(10,5))
plt.plot(epochs, acc, label='Training Accuracy') #color='lightblue',
plt.plot(epochs, val_acc, label='Validation Accuracy') #color='orange',
plt.title('Training and Validation Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
#plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.savefig('vgg_accuracy.png')

#loss plot
plt.figure(figsize=(10,5))
plt.plot(epochs, loss, color='pink', label='Training Loss')
plt.plot(epochs, val_loss, color='red', label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
#plt.ylim(ymin=0)
plt.legend(loc="best")
plt.grid(visible=True, which='major', color='#666666', linestyle='-')
plt.tight_layout()
plt.rcParams['axes.facecolor'] = 'white'
plt.savefig('vgg_loss.png')
```